

ROADRESQ

Deployment & DevOps Plan

Version 1.0 • Production Infrastructure & Delivery Baseline

Scope: Web • Mobile • API • PostgreSQL/PostGIS • Redis • WebSockets • Workers • Storage • CI/CD

Objective: Repeatable, secure, observable and recoverable production delivery

CODE → BUILD → TEST → DEPLOY → VERIFY → OBSERVE → ROLLBACK

Document Control

Field	Value
Document	Deployment & DevOps Plan
Product	RoadResQ
Version	1.0
Status	Production deployment baseline
Audience	Developers, QA, DevOps, security, product and operations
Primary goal	Reliable application delivery with controlled releases, infrastructure automation and fast recovery

1. Purpose

This document defines how RoadResQ is built, packaged, deployed, operated, monitored and recovered across development, CI, staging and production. It establishes a production-oriented DevOps baseline while keeping the initial architecture simple enough for a small engineering team.

1.1 DevOps Principles

- Infrastructure and deployment configuration should be version controlled.
- Every production artifact should be traceable to source code and a release version.
- Build once and promote the same tested artifact across environments where practical.
- Automate repeatable validation and deployment tasks.
- Use least privilege for users, services, CI runners and infrastructure.
- Prefer safe, reversible deployments over manual production changes.
- Treat observability, backup and recovery as part of the product—not optional add-ons.

2. Target Technology Stack

Layer	Recommended technology
Web frontend	Next.js + React + TypeScript + Tailwind CSS
Mobile	React Native + Expo
Backend	FastAPI + Python
Database	PostgreSQL + PostGIS
Cache / ephemeral state	Redis
Realtime	WebSockets
Background jobs	Python worker process + Redis-backed queue
Object storage	S3-compatible private bucket
Reverse proxy / ingress	Managed load balancer or Nginx/Traefik where self-managed
Containers	Docker
CI/CD	GitHub Actions or equivalent
Infrastructure	Cloud VM/container platform initially; managed services where operationally valuable
Observability	Centralized logs + metrics + traces/error tracking
External services	Maps, payments, OTP/SMS, push/email

3. Deployment Architecture

Recommended production topology:

Internet → DNS/CDN → Load Balancer/Ingress → Web/API → PostgreSQL/PostGIS + Redis → Workers/WebSockets → External Integrations

Static/media assets should use object storage/CDN where appropriate. The database remains the authoritative source for transactional state; Redis is used for ephemeral/cache/coordination workloads and must not be the only copy of critical business data.

3.1 Logical Components

- Web application: customer-facing web experience and operational/admin web interfaces.
- API application: authentication, business rules, bookings, providers, billing, support and admin APIs.
- Realtime gateway: authenticated WebSocket connections and event delivery.
- Worker service: asynchronous notifications, reconciliation, cleanup, retries and other background jobs.
- PostgreSQL/PostGIS: durable transactional and geospatial data.
- Redis: caching, rate limits, queues, short-lived locks and realtime coordination.
- Object storage: private provider documents, photos and other controlled media.
- Observability stack: logs, metrics, traces, error reporting and alerting.

4. Environment Strategy

Environment	Purpose	Deployment
Local	Developer implementation	Docker Compose or local services
CI	Automated validation	Ephemeral/test infrastructure
Staging	Production-like acceptance	Automated deployment from release candidate
Production	Real customer/provider operations	Protected deployment with approval

Environment configuration must be isolated. Production credentials, database URLs and payment secrets must never be copied into local development or CI logs.

5. Repository & Branching Strategy

Recommended repository layout:

apps/ web/ mobile/ api/ worker/ packages/ shared/ contracts/ infra/ docker/ migrations/ deployment/ docs/ tests/ .github/ workflows/

Recommended flow: feature/* → pull request → main/develop validation → staging → tagged release → production. The exact branch model may be simplified for a small team, but direct unreviewed production commits should be prohibited.

5.1 Git Rules

- Require pull requests for protected branches.
- Require automated checks before merge.
- Use conventional or consistently structured commit messages.
- Tag production releases using semantic versioning or an equivalent immutable release identifier.
- Never commit secrets, private keys, production database dumps or real customer/provider documents.

6. Containerization

Each backend runtime should have a deterministic container image. Separate runtime concerns where practical: API, worker and realtime gateway can use the same base image but different commands.

Example production process model:

```
api: uvicorn app.main:app --host 0.0.0.0 --port 8000 worker: python -m app.worker realtime: uvicorn app.realtime:app --host 0.0.0.0 --port 8001
```

- Use multi-stage builds where useful to keep runtime images small.
- Run as a non-root user.
- Pin major dependency versions and regularly update them.
- Add container health checks.
- Do not bake secrets into images.
- Generate SBOM/security scan results where supported.

7. CI Pipeline

Every pull request should run a predictable quality pipeline:

Checkout → Dependency install → Lint → Type check → Unit tests → Integration tests → Security/dependency scan → Build → API contract tests → E2E where appropriate

Gate	Purpose	Failure action
Lint/type check	Static correctness	Block merge
Unit tests	Business logic regression	Block merge
Integration tests	Dependency boundary validation	Block merge
Security scan	Known vulnerability detection	Block critical findings
Build	Artifact reproducibility	Block merge
Contract tests	API compatibility	Block merge
E2E smoke	Critical journey validation	Block release

8. CD Pipeline

Recommended release path:

Merge → Build immutable artifact → Deploy staging → Database migration validation → Smoke/E2E → Approval → Deploy production → Health checks → Monitor → Close release

- Production deployment should be performed by CI/CD or a controlled deployment mechanism—not manual file copying.
- Production deployment requires an identified version/commit.
- Migration steps must be explicitly ordered and logged.
- Post-deployment smoke tests should run automatically.
- Rollback criteria must be defined before deployment begins.

9. Secrets & Configuration Management

Configuration	Rule
Database credentials	Secret store/environment secret; never source-controlled
JWT/session secrets	Secret store with rotation procedure
Payment keys	Environment-specific secret store; production restricted

Configuration	Rule
Maps/API keys	Restrict by platform, origin and purpose where supported
Storage credentials	Least-privilege service identity
CI credentials	Short-lived/federated credentials preferred
Application config	Non-sensitive values may use version-controlled templates

Maintain a documented environment variable contract. Application startup should fail safely when mandatory production configuration is missing.

10. Database Deployment & Migrations

- Use versioned migrations for every schema change.
- Review migrations as code.
- Prefer backward-compatible migrations for zero/minimal-downtime releases.
- For destructive changes, use expand → migrate → contract.
- Take/verify backups before high-risk schema changes.
- Test migration execution on a production-like staging database.
- Record migration version and deployment identifier.
- Never edit production schema manually without an emergency change process and audit trail.

10.1 Safe Migration Pattern

RELEASE N: ADD NEW STRUCTURE → DEPLOY COMPATIBLE CODE → BACKFILL → SWITCH READ/WRITE → RELEASE N+1: REMOVE OLD STRUCTURE

11. Zero/Low-Downtime Deployment

- Run health checks before routing traffic to a new instance.
- Use rolling, blue/green or equivalent controlled deployment when supported.
- Keep application versions backward-compatible during rolling windows.
- Drain existing connections gracefully.
- Avoid schema changes that require simultaneous downtime unless explicitly approved.
- For WebSockets, support reconnect and authoritative state reconciliation after deployment.

12. Health Checks

Check	Purpose
/health	Basic process health
/ready	Dependency readiness before receiving traffic
Database connectivity	Detect DB access failures
Redis connectivity	Detect cache/queue dependency failures
Worker heartbeat	Confirm background processing
External integration health	Track critical provider availability without leaking credentials
Synthetic journey	Verify key customer/provider path at controlled intervals

13. Deployment Rollback

Rollback should be based on a predefined decision, not improvised during an incident.

Trigger	Example
Critical error increase	5xx rate or application errors materially exceed baseline
Booking integrity issue	Requests/assignments become inconsistent
Payment integrity issue	Incorrect payment state or duplicate financial effect
Realtime failure	Critical tracking/events unavailable at unacceptable level
Security issue	New release introduces exploitable behavior
Severe performance regression	Latency/resource saturation threatens service

Application rollback: route traffic to previous verified artifact. Database rollback: only when safe; otherwise use forward-fix migrations. Financial data should never be 'rolled back' by destructive database operations—use reconciliation/correction workflows.

14. Backup & Recovery Integration

- Automated PostgreSQL backups with defined retention.
- Point-in-time recovery where supported.
- Separate backup storage from the primary database environment.
- Encrypt backups at rest and in transit.
- Test restore procedures on a schedule.
- Back up configuration/critical infrastructure definitions as appropriate.
- Object storage must have its own backup/versioning/retention strategy where required.

Target RPO/RTO values must be formally selected by the business and documented in the Backup & Disaster Recovery plan.

15. Infrastructure as Code

Production infrastructure should be reproducible from version-controlled definitions. Terraform, Pulumi or an equivalent IaC tool may be adopted when infrastructure complexity justifies it.

- Version infrastructure definitions.
- Use separate state/workspaces/accounts/projects for environments.
- Protect infrastructure state and credentials.
- Require review for production infrastructure changes.
- Use plan/diff before applying changes.
- Document manual emergency changes and reconcile them back into code.

16. Networking & Security Baseline

- Expose only required public ports/services.
- Terminate TLS at a trusted edge/load balancer.
- Keep PostgreSQL and Redis private.
- Use security groups/firewall rules with least privilege.
- Restrict admin interfaces and privileged operational access.
- Use separate service identities for API, workers and deployment automation where practical.
- Enable audit logging for privileged infrastructure actions.

17. CDN, Storage & Media

- Store private provider documents in private object storage.
- Serve private files through short-lived signed URLs or an authorized download endpoint.
- Validate upload type, size and content before storage.
- Use lifecycle rules for temporary objects.
- Do not expose raw storage credentials to browsers or mobile clients.
- Use CDN caching only for content that is safe to cache publicly.

18. Scaling Strategy

Component	Initial scaling approach	Future path
Web	Horizontal instances/CDN	Autoscaling
API	Multiple stateless instances	Autoscaling + load balancing
Worker	Increase worker count/concurrency	Queue-based autoscaling
Realtime	Multiple gateway instances + shared coordination	Dedicated realtime tier
PostgreSQL	Vertical scale + indexing	Read replicas/partitioning when justified
Redis	Managed/sized instance	Cluster/sentinel where required
Storage	Managed object storage	CDN + lifecycle + replication

19. Observability & Alerting

- Structured application logs with request/correlation IDs.
- Metrics for traffic, errors, latency, database pool usage, Redis, queues and WebSockets.
- Distributed tracing for important cross-service operations where adopted.
- Error tracking with release/version tags.
- Alerts for availability, error spikes, queue backlog, DB health, resource saturation and payment/reconciliation anomalies.
- Dashboards for customer journey, provider operations and infrastructure.

Logs must not contain passwords, OTPs, payment secrets, authentication tokens or unnecessary sensitive location/contact data.

20. Deployment Observability Checklist

Signal	Post-release verification
5xx/error rate	No abnormal increase
Latency	Within expected baseline
CPU/memory	No unexpected saturation
DB	Connections/locks/errors healthy
Redis	Memory/latency/errors healthy
Queue	Jobs processing; no growing backlog
WebSockets	Connections/events healthy
Payments	No reconciliation anomalies

Signal	Post-release verification
Bookings	No abnormal cancellation/assignment patterns

21. CI/CD Security

- Protect CI secrets and deployment credentials.
- Use least-privilege tokens.
- Pin or verify third-party actions where practical.
- Run dependency and container vulnerability scans.
- Prevent untrusted pull requests from accessing production secrets.
- Require approval for production deployments.
- Audit deployment history and actor identity.
- Rotate credentials after suspected exposure.

22. Release Management

Release step	Required artifact
Planning	Scope + risk + owner
Implementation	Reviewed PRs
Validation	CI results + QA evidence
Staging	Deployment ID + smoke/E2E results
Approval	Explicit production approval
Production	Version + migration record
Verification	Health/metrics/smoke evidence
Close	Release notes + known issues + rollback status

23. Disaster / Major Incident Deployment Procedure

INCIDENT DETECTED → TRIAGE → STABILIZE → IDENTIFY LAST GOOD RELEASE → MITIGATE/ROLLBACK → VERIFY → COMMUNICATE → ROOT-CAUSE → RECOVERY ACTIONS

- Prioritize customer safety, booking integrity and payment correctness.
- Freeze unrelated deployments during a major incident.
- Assign incident commander/technical owner.
- Preserve logs and audit evidence.
- Use runbook-approved commands where possible.
- After stabilization, perform root-cause analysis and preventive action tracking.

24. Cost & Resource Controls

- Track compute, database, storage, bandwidth, logs and third-party API usage.
- Set budgets/alerts for cloud services.
- Avoid excessive high-frequency GPS writes; use appropriate sampling and retention.
- Use caching for safe high-read workloads.

- Archive/delete data according to approved retention policies.
- Review idle development/staging resources.

25. Production Access Model

Access level	Example permissions
Developer	Local/CI/staging; production read access only when justified
QA	Staging and test systems; controlled production verification
DevOps	Infrastructure/deployment administration
Security	Audit/security systems and incident access
Admin/Operations	Application-level operational permissions
Break-glass	Time-limited emergency access with mandatory audit

Use MFA for privileged accounts and prefer short-lived credentials. Production access should be attributable to an individual or controlled service identity.

26. Production Readiness Gates

- Application build is reproducible and versioned.
- All critical QA gates pass.
- Security baseline is satisfied.
- Database migrations have been tested.
- Backups exist and restore procedure is known.
- Health checks and monitoring are active.
- Alerts have been tested.
- Rollback path is documented and available.
- Secrets/configuration are environment-isolated.
- Operational runbook exists.
- Support/incident contacts and escalation path are defined.

27. Recommended Initial Infrastructure

For an MVP/early production launch, avoid unnecessary microservices. A practical deployment can use a managed PostgreSQL/PostGIS database, managed Redis, containerized FastAPI/API + worker + realtime processes, a managed web deployment or containerized Next.js app, private object storage, CDN/load balancer, and managed observability.

The architecture should remain modular so that high-load components can later be separated without rewriting core business domains.

28. Deployment Checklist

Phase	Checklist
Before merge	Code review, tests, security checks, migration review
Before staging	Build artifact created, config verified
Staging	Migration + smoke + E2E + integration validation
Before production	Approval, backup verification, rollback decision criteria

Phase	Checklist
Deploy	Controlled rollout, health/readiness checks
After deploy	Smoke tests, logs, metrics, queues, payments and bookings checked
Close	Release recorded, documentation updated, anomalies reviewed

29. Definition of Done — Deployment

- Deployment is automated and repeatable.
- Application artifacts are immutable/versioned.
- Environment configuration is isolated.
- Database migrations are versioned and tested.
- Health/readiness checks are implemented.
- Rollback strategy is documented.
- Logs, metrics and alerts are operational.
- Backups and recovery procedures are validated.
- Production access is controlled and auditable.
- Release evidence is retained.

30. Final DevOps Standard

RoadResQ should be operated as a production service from the beginning: version everything, automate what can be automated, protect production access, validate every release, monitor the customer/provider journey, and design every critical operation for failure and recovery. The initial implementation should remain operationally simple while preserving clear boundaries for future scaling.